

Pulling Tanzu RabbitMQ Images via the AceMQ JFrog Cache

Customer guide for AceMQ-licensed Tanzu RabbitMQ subscribers

Revised May 7, 2026

Overview

As an AceMQ customer with active commercial Tanzu RabbitMQ entitlements, you can pull Broadcom-published Tanzu RabbitMQ container images through AceMQ's JFrog Artifactory smart-remote repository. The repository fronts Broadcom's official registry (`rabbitmq.packages.broadcom.com`), caches images locally, and serves subsequent pulls at AceMQ's edge.

This guide covers the supported pull workflow for Docker, Podman, containerd, and Kubernetes. It applies to commercial Tanzu RabbitMQ versions on the 4.2.x, 4.1.x, and 4.0.x streams, and to the commercially supported community-edition maintenance series (3.13.x).

Repository facts at a glance

Registry hostname	<code>acemq.jfrog.io</code>
Docker remote repo	<code>rabbitmq-docker-remote</code>
Upstream registry	<code>rabbitmq.packages.broadcom.com</code>
Image path	<code>vmware-tanzu-rabbitmq</code>
Tag examples	<code>3.13.15</code> , <code>4.0.19</code> , <code>4.1.10</code> , <code>4.2.5</code>
Variant tag suffixes	<code>-arm64</code> , <code>-fips</code> , <code>-arm64-fips</code>

Prerequisites

- An active AceMQ account with access to the JFrog Artifactory instance.
- A valid Tanzu RabbitMQ commercial subscription or supported community-edition entitlement on file with AceMQ.
- A container client: `docker`, `podman`, or `nerdctl` (containerd).
- Outbound network access to `acemq.jfrog.io` on TCP/443. If you operate behind an egress firewall or proxy, allowlist that hostname and its CDN endpoints before the first pull.

Step 1 — Generate a JFrog Access Token

1. Log in to `https://acemq.jfrog.io`.
2. Click your username in the upper-right corner and select Edit Profile.

3. Open the Identity Tokens section and click Generate Token.
4. Set scope to Scoped Token with applied permissions read on rabbitmq-docker-remote (least-privilege). Set an expiration that matches your organization's secret-rotation policy.
5. Copy the token immediately. JFrog displays it only once.

Token handling

Treat the token as a credential. Store it in a secrets manager (HashiCorp Vault, AWS Secrets Manager, Azure Key Vault, Kubernetes Secret) and never commit it to source control. For CI/CD pipelines, use a service-account user in JFrog rather than a personal account.

Step 2 — Authenticate Your Container Client

Docker

```
docker login acemq.jfrog.io
Username: <your JFrog username>
Password: <paste the JFrog access token>
```

Podman

```
podman login acemq.jfrog.io
```

containerd / nerdctl

```
nerdctl login acemq.jfrog.io
```

Credentials are stored in your client's standard config file (~/.docker/config.json, \$XDG_RUNTIME_DIR/containers/auth.json) as a base64-encoded auth header. They are not encrypted at rest — protect the file accordingly.

Step 3 — Pull a Tanzu RabbitMQ Image

Pull a specific, pinned version. Always specify an explicit tag in production:

```
docker pull acemq.jfrog.io/rabbitmq-docker-remote/vmware-tanzu-
rabbitmq:3.13.15
```

For ARM64 hosts or FIPS-mode deployments, append the variant suffix:

```
# ARM64
docker pull acemq.jfrog.io/rabbitmq-docker-remote/vmware-tanzu-
```

```
rabbitmq:3.13.15-arm64
```

```
# FIPS (amd64)
```

```
docker pull acemq.jfrog.io/rabbitmq-docker-remote/vmware-tanzu-  
rabbitmq:3.13.15-fips
```

```
# FIPS on ARM64
```

```
docker pull acemq.jfrog.io/rabbitmq-docker-remote/vmware-tanzu-  
rabbitmq:3.13.15-arm64-fips
```

How smart-remote caching works

If the requested tag is not yet cached in JFrog, the first pull triggers an upstream fetch from Broadcom and may take longer. Once cached, the image is served from AceMQ's edge to all downstream clients. If another customer has already pulled that same tag, your first pull will be fast — the cache is shared at the repo level, not per user.

JFrog also re-validates upstream metadata on a configurable interval. If Broadcom replaces a mutable tag (rare for Tanzu releases, but possible for latest or branch tags), the cache will refresh on the next pull after the metadata TTL expires.

Step 4 — Discover Available Versions

Via the JFrog UI

Navigate to Artifacts → rabbitmq-docker-remote → vmware-tanzu-rabbitmq. Each tag listed corresponds to a pullable image.

Via the Docker Registry v2 API

List all tags for the image. Authentication is required — use your JFrog username and access token with HTTP Basic auth:

```
curl -u '<jfrog-username>:<access-token>' \
  https://acemq.jfrog.io/artifactory/api/docker/rabbitmq-docker-remote/  
v2/vmware-tanzu-rabbitmq/tags/list
```

This returns a JSON document of the form `{"name": "vmware-tanzu-rabbitmq", "tags": [...]}`. Pipe through `jq` for readable output:

```
curl -s -u '<user>:<token>' \
  https://acemq.jfrog.io/artifactory/api/docker/rabbitmq-docker-remote/  
v2/vmware-tanzu-rabbitmq/tags/list \
  | jq -r '.tags[]' | sort -V
```

Currently supported Tanzu RabbitMQ release streams

- **4.2.x** — current GA release stream. 4.2.5 is the latest patch.
- **4.1.x** — supported maintenance stream. 4.1.10 is the latest patch.
- **4.0.x** — supported maintenance stream. 4.0.19 is the latest patch.
- **3.13.x** — community-edition maintenance series with commercial support; 3.13.15 is the current OCI patch level (the Tanzu RabbitMQ for Kubernetes 3.13.x series is one patch behind at 3.13.14). Confirm support window with AceMQ before adopting for new deployments.

Open-source RabbitMQ 4.3.0 was released in April 2026, but Tanzu commercial 4.3 has not yet reached GA at the time of this revision. Confirm with AceMQ before pulling any 4.3 tag — it may not be covered under your commercial subscription.

Avoid the "latest" tag in production

Broadcom does not publish a stable `latest` tag for Tanzu RabbitMQ that crosses major version streams. Pulling `:latest` against this repository is unpredictable and can silently move you across major versions during routine rebuilds. Always pin to an explicit semver tag — or, even better, an immutable digest (see Step 6).

Step 5 — Kubernetes Deployment

Because `acemq.jfrog.io` is a private registry, Kubernetes requires an `imagePullSecret` before any pod can pull from it.

Create the pull secret

```
kubectl create secret docker-registry acemq-jfrog-pull \
  --namespace rabbitmq-system \
  --docker-server=acemq.jfrog.io \
  --docker-username=<jfrog-username> \
  --docker-password='<access-token>' \
  --docker-email=<your-email>
```

Reference it from a workload

In a Pod, Deployment, or RabbitMQ cluster CR, reference both the image and the secret:

```
spec:
  imagePullSecrets:
    - name: acemq-jfrog-pull
  containers:
    - name: rabbitmq
      image: acemq.jfrog.io/rabbitmq-docker-remote/vmware-tanzu-
rabbitmq:3.13.15
      imagePullPolicy: IfNotPresent
```

RabbitMQ Cluster Operator (RabbitmqCluster CR)

```

apiVersion: rabbitmq.com/v1beta1
kind: RabbitmqCluster
metadata:
  name: production
  namespace: rabbitmq-system
spec:
  image: acemq.jfrog.io/rabbitmq-docker-remote/vmware-tanzu-
rabbitmq:3.13.15
  imagePullSecrets:
    - name: acemq-jfrog-pull
  replicas: 3

```

Helm values override

If you install Tanzu RabbitMQ for Kubernetes via Helm, override the image and pull-secret values to point at the AceMQ cache:

```

rabbitmqImage:
  registry: acemq.jfrog.io
  repository: rabbitmq-docker-remote/vmware-tanzu-rabbitmq
  tag: 3.13.15
  pullSecrets:
    - acemq-jfrog-pull

```

Note that Tanzu RabbitMQ for Kubernetes also ships supporting components — the cluster operator, messaging-topology operator, audit-logger, and cert-manager. Each has its own image. If your installation pulls these from Broadcom directly, you may want to mirror them through `rabbitmq-docker-remote` as well so that the entire control plane is served from the cached registry. Contact AceMQ support to confirm what is in scope for your subscription.

Step 6 — Production Best Practice: Pin by Digest

Tags are mutable; digests are not. For audited or compliance-bound deployments, pin by SHA256 digest to guarantee bit-for-bit reproducibility.

Find the digest

After pulling the image, inspect it to retrieve the repository digest:

```

docker pull acemq.jfrog.io/rabbitmq-docker-remote/vmware-tanzu-
rabbitmq:3.13.15
docker inspect --format='{{index .RepoDigests 0}}' \
  acemq.jfrog.io/rabbitmq-docker-remote/vmware-tanzu-rabbitmq:3.13.15

```

The output looks like:

```
acemq.jfrog.io/rabbitmq-docker-remote/vmware-tanzu-rabbitmq@sha256:<64-hex-chars>
```

You can also retrieve the digest from the JFrog UI by navigating to the tag and copying the manifest SHA256 value.

Use the digest

```
docker pull acemq.jfrog.io/rabbitmq-docker-remote/vmware-tanzu-rabbitmq@sha256:abc123...
```

In Kubernetes manifests:

```
image: acemq.jfrog.io/rabbitmq-docker-remote/vmware-tanzu-rabbitmq@sha256:abc123...
```

Tag-and-digest hybrid

For human-readability plus immutability, you can specify both: `vmware-tanzu-rabbitmq:3.13.15@sha256:abc123...`. The container runtime ignores the tag for pull purposes and uses only the digest, but downstream tooling and humans can still read the version.

Troubleshooting

- Re-run docker login. Tokens expire if you set an expiration during creation.
- Confirm your AceMQ JFrog account has read permission on rabbitmq-docker-remote.
- If using a service account, verify the token was generated under the correct user.

'manifest unknown' or 'not found'

- Verify the tag exists upstream by querying the v2 tags/list endpoint (Step 4).
- Check that you are using the exact case and suffix — Tanzu variant suffixes are case-sensitive.
- Tags newly published to Broadcom may take a short interval to appear via the JFrog smart remote depending on the cache TTL configured.

Slow first pull, fast subsequent pulls

Expected behavior. The first pull of a previously uncached tag fetches from Broadcom; subsequent pulls hit the AceMQ-side cache.

'ErrImagePull' or 'ImagePullBackOff' on Kubernetes

- Confirm the `imagePullSecret` exists in the same namespace as the pod and is referenced in the pod spec.
- Inspect events: `kubectl describe pod <pod-name> -n <namespace>` — the events section shows the underlying registry error.
- Test the credentials manually from a node by running `docker login acemq.jfrog.io` with the same username/token used in the secret.

Egress / firewall failures

- Ensure outbound TCP/443 to `acemq.jfrog.io` is permitted.
- If using a corporate proxy, configure `HTTPS_PROXY` / `NO_PROXY` in your container client and on Kubernetes nodes.
- JFrog may serve image layers from a CDN endpoint distinct from the primary hostname. If layer downloads fail after a successful manifest pull, broaden allowlisting or contact AceMQ for the current CDN endpoint list.

Summary

1. Generate a scoped, expirable JFrog access token under your AceMQ account.
2. Authenticate your container client to `acemq.jfrog.io` using the token as the password.
3. Pull explicit tags from `acemq.jfrog.io/rabbitmq-docker-remote/vmware-tanzu-rabbitmq`, appending variant suffixes (`-arm64`, `-fips`) as required.
4. For Kubernetes, create a `docker-registry` pull secret in each namespace that runs RabbitMQ workloads and reference it in your pod or operator CR.
5. For production, pin images by SHA256 digest rather than tag.
6. Avoid the latest tag — Broadcom does not publish a stable cross-version latest for Tanzu RabbitMQ.

Support

For licensing questions, subscription scope, or registry access issues that this guide does not resolve, contact AceMQ support. For RabbitMQ broker behavior, configuration, and upgrade questions, AceMQ support engineers will engage Broadcom under your active subscription as needed.